TAMU DATATHON

Poker Bot Challenge

Participant Guide: Heads-Up Short Deck 5-Card Pot-Limit Omaha

Write a program that plays poker. Your Python bot will play every other team's bot in a round-robin tournament, thousands of hands per match, and the best bot wins.

This guide explains everything you need: the rules of the game, how your bot talks to the referee, how matches are scored, the limits your bot runs under, how to test it, and how to submit it. You do **not** need to know poker already. Section 3 teaches the game from scratch.

What	Where
Starter code (GitHub)	***https://github.com/tamu-datathon-org/heads-up-bundle***
MANDATORY TEAM REGISTRATION	*https://docs.google.com/forms/d/e/1FAIpQLSeVRUzERWDfrri24U6kA2tUbjT0530lwudYokp3ZpoxfXVAwA/viewform?usp=sharing&ouid=100967425964110292916*
Submission form	*https://forms.gle/a6Nr9VxXD9RYvi9x7*

NOTE: Your submission will not go through if you do not register! Registering the same person on multiple teams will result in invalid registration for the whole team!

Contents

1. The challenge at a glance
2. Getting started
3. The game: Short Deck 5-Card Pot-Limit Omaha
4. Writing your bot
5. Time limits, errors and forfeits
6. How matches and the tournament are scored
7. Where your bot runs in the tournament
8. Testing your bot locally
9. Submitting
10. Strategy ideas
11. Frequently asked questions
12. Quick reference

1. The challenge at a glance

You will build a **poker bot**: a small Python web server that receives a description of the game, decides what to do, and answers with an action. A referee program (the **judge**) deals the cards, asks each bot for its move, enforces the rules, and keeps score.

- **The game:** heads-up (two players) **Short Deck 5-Card Pot-Limit Omaha**. It's like Texas Hold'em, but with a 36-card deck, five private cards each, and a rule that you must use exactly two of them.
- **Your job:** fill in three Python functions in one file, `agent.py`. Everything else, including the web server, is already written for you.
- **The tournament:** every bot plays every other bot once. Each match is 2,000 hands, played so that both bots get exactly the same cards (Section 6).
- **Winning:** a match is won by a clear, statistically significant margin. Close matches are a draw (a "chop"). Most points wins the tournament.
- **Any approach is allowed:** hand-written rules, simulations, statistics, or machine learning. You choose the libraries.

> **The most important rule**
> If your bot crashes or breaks the rules too often during a match, your team **loses that match**. A simple bot that always works beats a clever bot that sometimes crashes. Test thoroughly (Section 8).

2. Getting started

2.1 Get the starter code

All starter code is in this GitHub repository:
https://github.com/tamu-datathon-org/heads-up-bundle

Copy it into a repository of your own team (for example with GitHub's **Use this template** or **Fork** button, or by cloning and pushing to a new repository). You will submit a link to **your** repository and a specific commit.

2.2 What's in the repository

File	What it is	Do you edit it?
`agent.py`	Your bot. A complete web server with three empty functions for your strategy.	**Yes**, between the YOUR CODE HERE markers
`game.py`	The rules: dealing, betting, legal actions, raise sizes, showdown. Read it to settle any rules question.	No
`evaluator.py`	Card helpers and a fast hand evaluator (`omaha_score`).	No
`judge_engine.py`	The referee. Run it locally to play your bot against another bot.	No
`baseline-bots/`	Simple opponents to test against, e.g. `baseline_random.py`.	No (copy them freely)
`test_agent.py`	Checks that the tournament can run your bot.	No

The tournament uses its own copies of `game.py`, `evaluator.py` and the judge, so changes you make to those files have no effect there. Only `agent.py` (and any files it imports) is yours.

2.3 Set up Python

You need **Python 3.10 or newer**. Install the two packages the bot's web server uses:

```
pip install fastapi uvicorn
```

Then check that everything runs. Open three terminal windows in the repository folder:

```
python agent.py                              # window 1: your bot (port 5008)
python baseline-bots/baseline_random.py      # window 2: an opponent (port 5009)
python judge_engine.py --deals 200           # window 3: the judge plays a match
```

The judge prints a report ending in a line such as `RESULT: AGENT2_WIN`. The starter bot only checks or folds, so expect it to lose. That's your starting point.

2.4 First edits

1. In `agent.py`, set `PARTICIPANT` to your team name and `AGENT_NAME` to a name for your bot.
2. Replace the example strategy in `choose_action()` with your own (Section 4).
3. Run `python test_agent.py` and a local match after every change.

3. The game: Short Deck 5-Card Pot-Limit Omaha

This section teaches the game from scratch. If you already play poker, skim it, but read **3.3** (hand rankings are different from normal poker), **3.4** (the Omaha rule) and **3.7** (bet sizes).

3.1 Words you'll see

Word	Meaning
Chips	Points used for betting. Every hand starts with each player holding **80 chips**.
Pot	All chips both players have put in during the current hand. The winner of the hand takes the pot.
Blinds	Forced bets before any cards are seen, so there is always something to win. The **small blind** is 1 chip and the **big blind** is 2 chips.
bb	"Big blinds". Results are measured in bb, so +1 bb means you won 2 chips. Each player starts every hand with 40 bb.
Button	The player who posts the small blind. The button alternates every hand.
Street	One round of betting. There are four: **preflop**, **flop**, **turn**, **river**.
Board	The five shared cards dealt face up in the middle, used by both players.
Hole cards	Your five private cards. Only you can see them.
Showdown	If nobody folds, both players reveal their cards and the better hand wins.
All-in	A player has put all their chips in. They can't bet further, and the remaining cards are dealt out.

3.2 The deck

Short Deck uses only **36 cards**: the ranks 6, 7, 8, 9, T (ten), J, Q, K, A in four suits (clubs c, diamonds d, hearts h, spades s). The 2s, 3s, 4s and 5s are removed. Cards are written as rank then suit, so `Ah` is the ace of hearts and `6c` is the six of clubs.

Inside your code, each card is an integer from 0 to 35:

```
rank = card >> 2     # 0 = six, 1 = seven, ..., 7 = king, 8 = ace
suit = card & 3      # 0 = clubs, 1 = diamonds, 2 = hearts, 3 = spades
```

You don't need to remember this: use `card_rank(c)`, `card_suit(c)` and `cards_to_str(cards)` from `evaluator.py`. For example, `cards_to_str([35, 0])` gives `"As 6c"`.

3.3 Hand rankings (different from normal poker!)

A poker hand is five cards. From strongest to weakest:

Rank	Hand	Example	Note
1 (best)	Straight flush	9h Th Jh Qh Kh	Five in a row, all one suit
2	Four of a kind	Q Q Q Q 7	
3	**Flush**	6s 9s Js Ks As	**Beats a full house** in Short Deck
4	Full house	T T T 8 8	Three of a kind plus a pair
5	**Three of a kind**	7 7 7 K J	**Beats a straight** in Short Deck
6	Straight	8 9 T J Q	Five in a row, mixed suits
7	Two pair	A A 9 9 6	
8	One pair	K K Q 8 6	
9 (worst)	High card	A Q T 8 7	Nothing else

Why the changes? With fewer low cards, flushes are rarer than full houses and straights are more common than three of a kind, so the rankings swap to keep rarer hands stronger.

The ace is high and low. The ace normally ranks above the king, but it can also act as the low card in the straight **A-6-7-8-9** (the missing 2-5 are skipped). That is the lowest straight: it loses to 6-7-8-9-T. The same goes for straight flushes.

Ties inside the same category are broken by the usual card-by-card comparison (higher pairs first, then the highest remaining cards, called "kickers"). Suits never break ties. Exactly equal hands split the pot.

3.4 The Omaha rule: exactly 2 + 3

At showdown, each player makes their best five-card hand using **exactly two** of their five hole cards and **exactly three** of the five board cards. Not one, not three, not four: always two from your hand and three from the board. That gives 10 × 10 = 100 possible hands, and the best one counts.

> **Example: the rule people get wrong**
>
> Board: `Ah Kh Qh Jh 6c`. You hold `Th 7s 7d 8c 9c`.
>
> Do you have a royal flush (`Ah Kh Qh Jh Th`)? **No.** That would use four board cards and only one hole card. A flush is impossible too: you'd need **two** hearts in your hand, and you only have one.
>
> Your best hand is `Th 9c` from your hand plus `Kh Qh Jh` from the board: 9-T-J-Q-K, a **king-high straight**.
>
> Lesson: lots of one suit on the board doesn't give you a flush unless **two** of your hole cards are that suit.

`omaha_score(hole_cards, board)` applies this rule for you and returns a number: **higher is better, equal is a tie**. It needs the full five-card board.

3.5 How one hand is played

1. **Setup.** Both players start with 80 chips (stacks reset every hand, so a bad hand never carries over). The button posts the small blind (1), the other player the big blind (2). Each player gets 5 hole cards.
2. **Preflop.** The **button acts first.** Betting continues until both players have acted and put in the same amount, or someone folds.
3. **Flop.** Three board cards are revealed. Another betting round. From now on the **non-button player acts first.**
4. **Turn.** A fourth board card. Another betting round.
5. **River.** The fifth board card. A final betting round.
6. **Showdown.** If nobody folded, the higher `omaha_score` wins the whole pot. Equal hands split it.

A hand ends early if someone **folds** (the other player wins the pot, and no cards are shown). If a player goes **all-in** and the bets are matched, there is no more betting: the rest of the board is dealt and the hand goes to showdown.

After a preflop call of the big blind, the big blind still gets to act (check or raise) before the flop. This is called the big blind's **option**.

3.6 The four actions

Action	What it does	When it's legal
`FOLD`	Give up the hand. The opponent wins the pot.	Only when you owe chips (`to_call > 0`). You can't fold when checking is free.
`CHECK_CALL`	If you owe nothing: **check** (pass). If you owe chips: **call** (match the bet). If you can't afford the full call, you go all-in with what you have.	Always
`RAISE_HALF`	Raise by about **half the pot**.	See below
`RAISE_POT`	Raise by the **size of the pot**, the most allowed ("pot-limit").	See below

Both raises are legal only if **all** of these are true:

- Fewer than **4 raises** have happened on this street (blinds don't count). After 4, players can only call or fold.

- You have more chips behind than you need to call (`my_stack > to_call`).
- Your opponent is not already all-in.

Always pick from `obs.legal_actions`. It lists exactly what you may do right now.

3.7 How big is a raise?

You never choose an amount. You choose half-pot or pot, and the game works out the amount. Using the values from your observation:

```
pot_raise_to  = opp_bet + pot + to_call             # the pot-limit maximum
half_raise_to = opp_bet + (pot + to_call) // 2
min_raise_to  = opp_bet + max(last raise size, 2)    # a raise must be at least this big

raise_to = min( max(target, min_raise_to), pot_raise_to, my_bet + my_stack )
```

"Raise to" means your **total** bet on this street afterwards. In words: your raise is at least the minimum raise, at most a pot-sized raise, and never more than all your chips (if you're short, the raise becomes an all-in). You don't need to compute any of this yourself: `obs.raise_to[Action.RAISE_HALF]` and `obs.raise_to[Action.RAISE_POT]` already hold the exact totals.

Worked example

The button (player A) has posted 1 and player B has posted 2, so the pot is 3.

#	Situation	Choice and result
1	Preflop, A to act. To call = 1. Half-pot raise goes to 4, pot raise to 6.	A plays `RAISE_POT`: A's bet is 6.
2	B to act. To call = 4, pot = 8. Half-pot → 12, pot → 18.	B plays `RAISE_POT`: B's bet is 18.
3	A to act. To call = 12.	A calls 12. Each has put in 18; pot = 36; each has 62 left.
4	Flop, B acts first. To call = 0. Half-pot → 18, pot → 36.	B plays `RAISE_POT` (a bet of 36).
5	A to act. To call = 36, pot = 72, A has 62 left. A pot raise would be 144, but A only has 62.	Both raises are an **all-in** for 62.
6	B to act. To call = 26 and B has exactly 26 left, so B can't raise.	B may only `FOLD` or `CHECK_CALL`. If B calls, the turn and river are dealt and the hand goes to showdown.

Notice how fast the pot grows: two players can be all-in on the flop. With only 40 big blinds each, every pot-sized raise is a big decision.

4. Writing your bot

Your whole bot lives in `agent.py`. The bottom half of the file (marked **SERVER PLUMBING**) is the web server that talks to the judge. **Don't change it**: if it breaks, the judge can't talk to your bot. You fill in three functions between `# ===== YOUR CODE HERE =====` and `# ===== END OF YOUR CODE =====`.

4.1 The three functions

Function	When it's called	What you do
`new_half(config, memory)`	Once at the start of each half-match (1,000 hands).	Set up anything you want to track, e.g. `memory["opp_raises"] = 0`.
`choose_action(obs, memory)`	Every time it's your turn.	Look at `obs` and **return one of `obs.legal_actions`**.
`on_hand_end(result, memory)`	Once after every finished hand.	Optional: learn from what happened (e.g. count how often the opponent raises).

Here is a minimal example that shows how the pieces fit together. It always returns a legal action, but it is **not** a good strategy: in our tests it loses to the random baseline bot. Beating it is your first goal.

```
def choose_action(obs: Observation, memory: dict) -> Action:
    # Before the river we can't score our hand yet: only call cheap bets.
    if obs.street != Street.RIVER:
        if obs.to_call <= obs.big_blind:          # cheap: stay in the hand
            return Action.CHECK_CALL
        if Action.FOLD in obs.legal_actions:      # too expensive: give up
            return Action.FOLD
        return Action.CHECK_CALL

    # On the river all 5 board cards are out, so we can score our hand.
    score = omaha_score(obs.hole_cards, obs.board)
    if score >= (FLUSH << 20) and Action.RAISE_POT in obs.legal_actions:
        return Action.RAISE_POT                   # very strong: bet big
    return Action.CHECK_CALL
```

(This needs `from evaluator import FLUSH` and `from game import Street` at the top. Hand scores keep the hand category in their top bits, so `score >> 20` gives the category from 0 = high card to 8 = straight flush. `category_name(score)` turns it into text like `"flush"`.)

4.2 What your bot can see: the Observation

`choose_action` receives an `Observation` with these fields. All chip amounts are whole numbers.

Field	Meaning
`hand_index`	Which hand of the half this is: 0, 1, 2, ... 999.
`street`	`Street.PREFLOP`, `Street.FLOP`, `Street.TURN` or `Street.RIVER`.
`hole_cards`	Your 5 cards as integers, sorted from low to high.
`board`	The board cards shown so far: 0 (preflop), 3 (flop), 4 (turn) or 5 (river).
`pot`	All chips both players have put in this hand, blinds included.
`to_call`	Chips you must add to match your opponent. 0 means you can check.
`my_bet`, `opp_bet`	Chips you and your opponent have put in **on this street** (blinds count preflop).
`my_stack`, `opp_stack`	Chips each player still has behind (not yet bet).
`is_button`	`True` if you are the button (small blind) this hand.
`legal_actions`	The actions you may return right now. **Always return one of these.**

Field	Meaning
`raise_to`	For each legal raise, your total street bet after that raise, e.g. `{RAISE_HALF: 12, RAISE_POT: 18}`.
`min_raise_to`	The smallest normal raise total (an all-in can be smaller).
`raises_this_street`	Raises so far on this street (maximum 4).
`action_history`	Every action this hand so far, oldest first. Each has `street`, `is_me` (True if it was yours), `action`, `amount` (chips that action added) and `street_total`. Blinds are not listed.
`big_blind`	Size of the big blind in chips (2).

4.3 What you learn after a hand: the HandResult

Field	Meaning
`hand_index`	Which hand it was.
`my_net`	Chips you won (positive) or lost (negative) in that hand.
`pot`	The final pot size.
`went_to_showdown`	`True` if cards were revealed (nobody folded).
`board`	The board cards that were revealed. After a fold, only the cards shown by then.
`opp_hole`	Your opponent's 5 cards, **only after a showdown**; otherwise `None`.
`winner`	`"me"`, `"opp"` or `"split"`.

To save time, finished hands are delivered together with your next decision, so `on_hand_end` always runs before `choose_action` sees the next hand. The last few hands of a half are delivered when the half ends.

4.4 Remembering things: use memory

Store everything you want to remember in the `memory` dictionary. It is **wiped clean at the start of each half**. In the tournament, each half also runs in a **brand-new copy of your server**, so nothing at all (not memory, not global variables, not files) carries over from one half to the other. Use `memory` and your bot behaves the same locally and in the tournament.

Reading files that are part of your repository (for example a trained model loaded when the file starts) is fine. See Section 7 for writing files.

4.5 Rules for your code

- Always return a value from `obs.legal_actions`. Anything else counts as an error (Section 5).
- Keep `choose_action` fast. You share a small time budget across a whole half (Section 5).
- Your server answers one request at a time, so your functions never run twice at once.
- Catch your own mistakes: wrapping your strategy in `try/except` and falling back to `CHECK_CALL` is a cheap way to avoid errors.
- Don't try to interfere with the judge, the other bot, or the machine. Bots that do will be disqualified.

5. Time limits, errors and forfeits

5.1 The time bank

Your bot has a **time bank of 8 seconds of thinking time per half** (1,000 hands). Every request the judge sends you is timed, and the time is taken from your bank. The first millisecond of each request is free, to cover network delay.

- A single request may take at most **1 second**. Taking longer counts as an error.
- When your bank runs out, your bot is no longer asked: for the rest of the half it automatically **checks when that's free and folds otherwise**. That's not an error, but it will lose you a lot of chips.

> **Do the maths**
> A half has 1,000 hands and you typically decide 2 to 4 times per hand, so 8 seconds means roughly **2 to 4 milliseconds per decision on average**. A full board evaluation with `omaha_score` takes a few microseconds, so you can afford some simulation, but not thousands of full simulations on every decision. Measure your bot's time with the judge's report (Section 8).

5.2 What counts as an error

Each of these counts as **one error**, and the judge plays the default action for you instead (check if free, otherwise fold):

- Your code raises an exception (in `choose_action`, `on_hand_end` or `new_half`).
- You return something that isn't an action, or an action that isn't legal right now.
- A request takes longer than 1 second.
- Your server can't be reached or sends back a malformed reply.

5.3 Forfeits mean a loss

After **5 errors in one half**, your bot **forfeits** that half: it is never asked again, and the judge plays check-or-fold for it until the half ends. A bot also forfeits if it fails to start or doesn't answer at the beginning of a half.

> **In the tournament, a forfeit loses the match**
> If your bot forfeits **either half** of a match, your team gets a **loss (0 points)** for that match, however many chips you were winning by. Your opponent gets the win. If both bots forfeit, both teams get a loss.
> Running out of time bank is **not** a forfeit (but it's very costly, see 5.1).

The judge's report shows your error count, and for crashes it prints your bot's full Python traceback, so you can find and fix the bug.

6. How matches and the tournament are scored

6.1 Duplicate format: same cards for both bots

Poker has a lot of luck: a weak bot can win a match just by being dealt better cards. To cancel most of that luck, every match is played in **duplicate**:

- **Half 1:** 1,000 hands. Your bot sits in one seat, your opponent in the other.

- **Half 2:** the **same 1,000 deals** again, with the seats swapped. You now get exactly the cards your opponent had, and the same boards.

So over the whole match both bots hold exactly the same cards. Whoever plays them better comes out ahead. Because each half runs in a fresh copy of your bot, you can't remember the cards from the first half.

6.2 Who wins a match

For each of the 1,000 deals, the judge adds up your result from both halves, in big blinds. It then takes the **average per deal** and a **95% confidence interval** around it (average ± 1.96 × standard error).

Confidence interval	Verdict	Points
Entirely **above** 0	You win	You 1, opponent 0
Entirely **below** 0	You lose	You 0, opponent 1
Includes or touches 0	Chop (draw): too close to call	0.5 each

In plain words: you only win a match if the judge is confident your edge is real and not luck. Beating a weak bot by a lot is a sure win; two evenly matched bots will usually chop. And remember: a forfeit is always a loss (Section 5.3).

6.3 The tournament

Every bot plays every other bot exactly once (a round robin). Standings are ordered by **total points**. Ties on points are broken by **average big blinds won per deal** across all your matches.

7. Where your bot runs in the tournament

Your repository is built into a container image, and that image is what plays. Design your bot for these conditions:

Limit	Value	What it means for you
CPU	1 CPU core	Heavy multi-threading won't help. Keep decisions cheap.
Memory	4 GB	Your model and data must fit, with room to spare.
Internet	None	Your bot can't download anything while playing. Everything it needs must be in your repository.
Files	Read-only, except `/tmp`	Your bot can read its own files. It can only write to `/tmp` (512 MB limit), which is empty at the start of each half.
User	Not administrator (root)	Don't rely on administrator rights.
Image size	3 GB added on top of the base image	Very large models or datasets may not fit.
Build time	30 minutes	Installing your packages must finish within this.
Fresh start	New server process every half	No state carries over between halves (Section 4.4).

7.1 Libraries

There are **no restrictions** on which libraries you use. The base image already includes Python, the packages the starter bot needs, and **PyTorch (CPU version)**. GPUs are not available, so CPU-only machine learning is the way to go.

Declare your additional requirements in the requirements.txt. Ensure this file is in the root of your repository.

7.2 Build check

When you submit, your bot is built and given a quick **smoke test** under exactly the limits above. If the build or smoke test fails, that submission is rejected (and if you had an earlier working submission, it stays in the tournament). Run `test_agent.py` before submitting to catch problems early.

8. Testing your bot locally

8.1 Play a match

Run your bot and an opponent in two windows, then the judge in a third:

```
python agent.py                              # your bot, port 5008
python baseline-bots/baseline_random.py      # opponent, port 5009
python judge_engine.py                       # 1,000 deals, both halves
```

Useful judge options:

Option	What it does
`--deals 200`	Play fewer deals per half, for a quicker test.
`--seed 7`	Change the cards. The same seed always deals the same cards, so you can reproduce a match exactly.
`--json out.json --logs`	Save the full result, including every hand, to a file you can study.
`--agent1 5010 --agent2 5011`	Use other ports, e.g. to play two versions of your own bot against each other.

To run a second bot on another port: `PORT=5010 python agent.py` (on Windows PowerShell: `$env:PORT=5010; python agent.py`).

8.2 Reading the report

- **agent1: +0.523 bb/deal**: agent1's average result per deal (agent1 is the bot on port 5008 by default).
- **Confidence interval** and **VERDICT**: who would win this match in the tournament.
- **Faults**: for each bot and half, thinking time used out of 8 s, number of calls and errors, and flags such as `FORFEITED` or `TIME BANK EMPTY`. If your bot crashed, its first error and traceback are printed below.
- **RESULT:** the last line: `AGENT1_WIN`, `AGENT2_WIN` or `CHOP`.

8.3 Checklist before you submit

1. `python test_agent.py` passes.

2. A full 1,000-deal match finishes with **0 errors** and no forfeits for your bot, against several opponents and several seeds.
3. Your bot's thinking time is comfortably under 8 s per half. Leave a safety margin: the tournament machine may be slower than your laptop.
4. Your bot beats the baseline bots.
5. Everything your bot needs (code, model files, data) is committed and pushed to your repository.

9. Submitting

Submit through the form: **[[Submission form link]]**. It asks for three things:

Field	Example	Notes
Team name	Ace High	Use the same name every time you resubmit, so your new bot replaces the old one.
GitHub repository URL	https://github.com/jdoe/poker-bot	Your team's repository, in exactly this form. It must be public.
Commit SHA	3f9a1c0e8b7d6a5f4e3d2c1b0a9f8e7d6c5b4a39	The **full 40-character** commit ID to build. Short IDs are rejected.

To find the full commit SHA of your latest commit, push your work and run:

```
git push
git rev-parse HEAD
```

Or on GitHub, open the commit and copy its full ID. The commit you submit must be pushed to GitHub, otherwise it can't be built.

9.1 Resubmitting

- You may resubmit before the deadline. Your **latest successful** build is the one that plays.
- If a new submission fails to build or fails the smoke test, your **previous working bot is kept**.
- Submitting the same commit twice changes nothing.

Deadline: **[[Date and time]]**.

10. Strategy ideas

You don't need any of these to take part. They're starting points if you're not sure where to begin.

- **Beat the baselines first.** A bot that folds weak hands, calls with decent ones and raises strong ones already beats random play by a wide margin.
- **Estimate your chances.** Before the river, deal the missing board cards (and a random opponent hand) many times and count how often you win with `omaha_score`. This is called **Monte Carlo equity**. Watch your time budget!
- **Compare price and chances.** If you must call 10 chips into a pot that will be 40, you need to win more than 25% of the time to profit.
- **Preflop shortcuts.** Precompute simple features of your five hole cards (pairs, suited cards, connected ranks, high cards) instead of simulating every preflop decision.

- **Position matters.** After the flop the button acts last and sees what the opponent did first. That's an advantage.
- **Model your opponent.** Use `on_hand_end` and `action_history` to track how often your opponent raises, folds or bluffs (cards revealed at showdown are gold), then adapt.
- **Machine learning.** You can train offline (self-play or simulation) and ship the trained model in your repository. Remember the 1-CPU, 4 GB, no-internet limits, and keep inference fast.
- **Robustness beats brilliance.** One unhandled exception path, triggered 5 times, loses the whole match.

11. Frequently asked questions

Do stacks carry over from hand to hand?

No. Both players start every hand with 80 chips. Your score is the sum of chips won or lost over all hands.

Can my bot see the opponent's cards?

Only after a showdown, in `on_hand_end` (`result.opp_hole`). Never during a hand.

Who is the button?

It alternates every hand. Check `obs.is_button`.

Can I choose any bet size?

No. Only the four actions exist. The game computes raise amounts (Section 3.7).

Why did my match end in a chop even though I won more chips?

Your winning margin wasn't big enough, compared with the randomness of the cards, for the judge to be 95% confident. Win by more, or more consistently.

Can I use several files or folders?

Yes. `agent.py` can import your own modules and load data files from your repository.

My bot works locally but failed the build check.

Usually a missing package, a file that wasn't committed, writing outside `/tmp`, using more than 4 GB of memory, or needing the internet. Check each against Section 7.

Can I edit the server plumbing or the judge?

Changing `game.py`, `evaluator.py` or `judge_engine.py` has no effect in the tournament, which uses its own copies. Changing the plumbing in `agent.py` risks breaking the connection to the judge, so please don't.

Are the numbers in this guide final?

The settings in this guide (stacks, blinds, time bank and so on) may be tuned before the tournament. If they change, the updated `game.py` in the starter repository is the source of truth, and we'll announce it.

12. Quick reference

Setting	Value
Deck	36 cards (6 through A, four suits)

Setting	Value
Hole cards / board	5 each / 5 shared cards
Making a hand	Exactly 2 hole cards + exactly 3 board cards
Blinds	1 (small, button) / 2 (big)
Starting stack	80 chips (40 bb), reset every hand
Raise cap	4 raises per street
Hands per match	2,000 (1,000 deals × 2 halves, seats swapped)
Time bank	8 seconds of thinking per half; 1 second max per request
Error limit	5 errors in a half = forfeit = match loss
Match verdict	95% confidence interval on bb/deal; win 1, chop 0.5, loss 0
Bot resources	1 CPU, 4 GB memory, no internet, writable /tmp only
Image size limit	3 GB on top of the base image